

Understanding the Dashboard, Model Training & Model Evaluation

AI-POWERED ZERO TRUST NETWORK INTRUSION DETECTION SYSTEM PHASE 1 COMPLETE STUDY HANDBOOK & VIVA MANUAL

Target Audience: Final-Year Computer Science & Cybersecurity Engineering Student

Purpose: Master Project Architecture, Dashboard, Training & Evaluation Screens for Viva Defense

Includes: 17 Detailed Chapters, Line-by-Line Code Breakdowns, 100 Viva Q&As, & Quick Revision Notes

Academic Year: 2026

Chapter 1: Introduction & System Philosophy

1.1 What is a Network Intrusion Detection System (NIDS)?

Explanation: A Network Intrusion Detection System (NIDS) is a security software module that monitors network traffic flow and analyzes patterns to identify malicious activity such as unauthorized port scans, flooding, or data theft attempts.

Real-World Analogy: Imagine an airport security scanner. It scans every piece of luggage going through the x-ray conveyor belt to detect prohibited items before passengers reach the gate.

1.2 What is Zero Trust Architecture?

Explanation: Zero Trust is a cybersecurity framework based on the principle *'Never Trust, Always Verify'*. In traditional networks, anyone inside the internal network was trusted automatically. Zero Trust requires identity verification and role permission checks on every single HTTP request.

1.3 Why Artificial Intelligence (AI) is Used

Explanation: Traditional NIDS systems use static IF-ELSE rules. Attackers can easily bypass static rules by modifying packet headers slightly. Deep learning AI models (like LSTMs) learn high-level mathematical patterns, allowing them to detect novel, previously unseen zero-day attacks.

1.4 How the Three Screens Fit Together

- **Screen 1 (Dashboard & Audit Trail):** Displays system health, active dataset, model status, and logs all security events.
- **Screen 2 (Model Training Interface):** Configures hyperparameters, executes dataset preprocessing, and trains the LSTM neural network with real-time status polling.
- **Screen 3 (Model Evaluation & Verification Report):** Analyzes model test accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC Curve AUC.

Chapter 2: Complete Project End-to-End Workflow

```
+-----+ +-----+ +-----+ | 1. User Sign-In | ---> | 2.
Auth Middleware | ---> | 3. SOC Dashboard | | (JWT Token Issued) | | (Bcrypt & JWT Check) | | (KPI Cards &
Audit) | +-----+ +-----+ +-----+ | v
+-----+ +-----+ +-----+ | 6. LSTM Training | <--- | 5.
Data Preprocess | <--- | 4. Select Dataset | | (64 Units, Epochs) | | (Scale & 3D Reshape) | | (NSL-KDD /
UNSW) | +-----+ +-----+ +-----+ | v
+-----+ +-----+ +-----+ | 7. Model Evaluation | ---> |
8. Save Artifacts | ---> | 9. Audit Log Entry | | (Confusion & ROC) | | (.keras, .pkl, .json)| | (Recorded in
SQLite) | +-----+ +-----+ +-----+
```

Chapter 3: Dashboard Architecture & Controls

The Executive SOC Dashboard ([templates/dashboard.html](file:///c:/Users/hp/Desktop/ML/templates/dashboard.html)) provides a full-width overview of system health:

Dashboard Element	UI Purpose & Description	Backend Data Source
Top Navigation Bar	Sticky header with links: Dashboard, Datasets, Compare, Training, Reports, Audit Logs.	Flask endpoint routing via `navbar.html`.
Uploaded Datasets Card	Displays total number of datasets registered in catalog.	<code>`DatasetHistory.query.count()`</code>
Active Dataset Card	Shows name and row/col count of selected training dataset.	<code>`DatasetHistory.query.filter_by(is_selected=True)`</code>

Model Status Card	Indicates whether an LSTM model has been trained ('Trained' / 'Not Trained').	<code>`ModelHistory.query.first()`</code> check.
Training Accuracy Card	Displays evaluation accuracy percentage of current trained model.	<code>`ModelHistory.accuracy * 100`</code>
Admin Portal Badge	Red badge identifying Admin role privileges vs blue User Portal badge.	<code>`g.user.role`</code> from JWT payload.
Logout Button	Clears JWT session token from browser localStorage and redirects to <code>`/login`</code> .	<code>`auth_routes.py`</code> logout API.

Chapter 4: Security Audit Trail & Event Logging

4.1 Audit Trail Table Schema

The Security Audit Trail records every administrative and security event into SQLite. Table columns:

- **Time:** UTC timestamp (``%Y-%m-%d %H:%M:%S``).
- **User:** Username of the operator who executed the action.
- **Action:** System action code (``MODEL_TRAINING_STARTED``, ``DATASET_PREPROCESSED``, ``DATASET_SELECT``, ``DATASET_DELETE``).
- **Status:** Success/Failure status (``SUCCESS``, ``FAILED``, ``ACCESS_DENIED``).
- **IP Address:** Remote IP address of client browser.
- **Details:** Context description string.

4.2 Flask Audit Log Code Snippet with Line-by-Line Breakdown

```
def log_audit_event(action, status, user_id, username, ip_address, details):
    log = AuditLog( action=action,
                    status=status, user_id=user_id, username=username, ip_address=ip_address, details=details )
    db.session.add(log)
    db.session.commit()
```

- `log = AuditLog(...)`: Instantiates a new ORM object matching the `audit_logs` table schema.
- `db.session.add(log)`: Stages the event log record inside SQLAlchemy session transaction.
- `db.session.commit()`: Persists the transaction permanently into the SQLite database file.

Chapter 5: Dataset Workflow & Catalog Management

5.1 Dataset Operations

- **Upload:** Ingests custom CSV datasets into ``datasets/raw/`` via ``/upload-dataset``.
- **Inventory:** Lists all registered datasets with row/col counts and file sizes at ``/dataset-list``.
- **Selection:** Sets ``is_selected=True`` for target dataset via ``/api/dataset/select``.
- **Delete:** Admin-only removal of dataset records and physical files via ``/api/dataset/delete/``.
- **Compare:** Side-by-side comparison matrix of datasets at ``/dataset-compare``.

Chapter 6: Model Training Interface Architecture

The Model Training Interface (`[templates/train_model.html](file:///c:/Users/hp/Desktop/ML/templates/train_model.html)`) consists of:

1. **Hyperparameter Panel:** Inputs for Epochs (10), Batch Size (64), Learning Rate (0.001), Optimizer (Adam), and Target Dataset dropdown.
2. **Real-Time Training Progress:** Displays progress bar (0% - 100%), epoch counter, accuracy/loss badges, and status text.

- 3. **Training Graph:** Live updating Chart.js line plot displaying Train Loss, Val Loss, Train Acc, and Val Acc across epochs.
- 4. **Latest Evaluation Report Button:** Appears upon completion to redirect user to `/model-report``.

Chapter 7: Hyperparameters Explained

Hyperparameter	Definition & Plain English Explanation	Default Value & Why Used
Epochs	One complete pass through the entire training dataset.	`10` — Allows model to learn weights without overfitting.
Batch Size	Number of training samples processed before updating weights.	`64` — Balances memory efficiency and gradient stability.
Optimizer	Algorithm used to adjust weight values to minimize loss error.	`Adam` — Adaptive Moment Estimation combining momentum and RMSProp.
Learning Rate	Step size taken by optimizer during gradient descent updates.	`0.001` — Optimal step size preventing gradient divergence.
Loss Function	Mathematical function measuring error between predictions and target labels.	`Categorical Crossentropy` — Measures loss across multi-class probability outputs.

Chapter 8: LSTM Neural Network Architecture from Scratch

8.1 Deep Learning Concepts

- **Artificial Intelligence (AI):** Broad discipline of making machines simulate human intelligence.
- **Machine Learning (ML):** Subfield where algorithms learn statistical patterns from data.
- **Deep Learning (DL):** Subset of ML utilizing multi-layered artificial neural networks.
- **LSTM:** Long Short-Term Memory network equipped with Input, Forget, and Output gates to remember temporal time-series patterns.

8.2 Neural Network Code Breakdown

```
model = Sequential([ LSTM(64, input_shape=(1, features), return_sequences=False), Dropout(0.2), Dense(32, activation='relu'), Dense(num_classes, activation='softmax') ])
```

- `Sequential([...])`: Defines a linear stack of neural layers.
- `LSTM(64, input_shape=(1, features))`: 64 unit LSTM cell extracting sequential time-series patterns.
- `Dropout(0.2)`: Drops 20% of hidden neurons randomly during training to prevent overfitting.
- `Dense(32, activation='relu')`: Fully-connected layer applying Rectified Linear Unit activation.
- `Dense(num_classes, activation='softmax')`: Output classification layer producing probability scores.

Chapter 9: Real-Time Training Progress & Graph Analysis

9.1 Identifying Training Behavior

- **Good Training:** Training Loss and Validation Loss decrease steadily together; Training Accuracy and Validation Accuracy increase together.
- **Overfitting:** Training Loss decreases near 0 while Validation Loss starts increasing; Training Accuracy is 99% while Validation Accuracy drops.
- **Underfitting:** Both Training Loss and Validation Loss remain high and flat across all epochs.

Chapter 10: Model Evaluation Report & Mathematics

Metric Name	Mathematical Formula	Simple English Interpretation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall percentage of correctly classified normal and attack samples.
Precision	$\frac{TP}{TP + FP}$	Out of all alerts flagged as attacks, what fraction were actual attacks?
Recall	$\frac{TP}{TP + FN}$	Out of all real attacks in dataset, what fraction did the model detect?
F1-Score	$2 * \frac{P * R}{P + R}$	Harmonic mean balancing Precision and Recall.
AUC (ROC)	Area under ROC curve plot	Model's capability to discriminate between normal and attack classes (1.0 = Perfect).

Chapter 11: Confusion Matrix Analysis & Class Imbalance

11.1 Matrix Structure

- **True Positive (TP):** Actual Attack correctly predicted as Attack.
- **True Negative (TN):** Actual Normal correctly predicted as Normal.
- **False Positive (FP):** Actual Normal incorrectly predicted as Attack (False Alarm).
- **False Negative (FN):** Actual Attack incorrectly predicted as Normal (Missed Threat!).

Viva Note — Why might a model predict attack classes as Normal?

Answer: Class Imbalance! If 95% of training samples are Normal and only 5% are Attack, the neural network learns that guessing 'Normal' minimizes cross-entropy loss. Solutions include SMOTE oversampling, class weighting, or gathering balanced datasets.

Chapter 12: ROC Curve & Area Under Curve (AUC)

The **ROC Curve (Receiver Operating Characteristic)** plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)** at various probability thresholds.

- **AUC = 1.0:** Perfect model with 100% true positive detection and 0% false alarms.
- **AUC = 0.5:** Random guessing (diagonal baseline).

Chapter 13: Interpreting Results & System Improvement

How to evaluate your model performance:

- **Accuracy > 95%, F1-Score > 0.90:** High performance model ready for deployment.
- **High Accuracy, Low Recall:** Model is suffering from class imbalance. Apply class weights in Keras (`class_weight={0: 1.0, 1: 5.0}`).
- **High Loss, Low Accuracy:** Increase training epochs (from 10 to 25) or add more units to LSTM layer.

Chapter 14: Phase 1 Serialized Output Files

File Name & Location	What it Contains / Format	How Phase 2 Uses It
----------------------	---------------------------	---------------------

models/nids_lstm_model.keras	Trained Keras LSTM neural network binary weights and architecture topology.	Phase 2 loads model via <code>`keras.models.load_model()`</code> to predict attacks on live packets.
models/scaler.pkl	Fitted Scikit-Learn <code>`StandardScaler`</code> object serialized via Python pickle.	Phase 2 normalizes live packet feature vectors using <code>`scaler.transform()`</code> .
models/label_encoder.pkl	Fitted <code>`LabelEncoder`</code> object.	Phase 2 maps text labels to numerical integers.
reports/latest_evaluation_report.json	JSON file storing test accuracy, precision, recall, f1, and confusion matrix.	Loaded by <code>`/model-report`</code> web view to render dashboard performance graphs.
database/database.db	SQLite relational database file.	Stores user accounts, dataset history, model runs, and audit logs.

Chapter 15: Internal Working & System Pipeline

- User Login:** User enters credentials at `/login`. Server verifies password via Bcrypt and returns JWT token.
- JWT Storage:** Token saved in browser `localStorage` and appended to Authorization header on every request.
- Dashboard Load:** User lands at `/`. Flask reads dataset count and audit logs from SQLite.
- Dataset Upload:** Admin uploads dataset CSV at `/upload-dataset`; metadata saved to `dataset_history` table.
- Dataset Selection:** Admin selects dataset at `/dataset-list`; sets `is_selected=True` in SQLite.
- Training Trigger:** Admin clicks 'Start Training Model' at `/train-model`. Flask starts background training thread.
- Data Preprocessing:** Data loaded, median imputed, LabelEncoded, StandardScaler normalized, and reshaped to 3D tensor.
- LSTM Model Compilation:** 64-unit LSTM model compiled with Adam optimizer and Categorical Crossentropy loss.
- Training Epochs:** Model trains for 10 epochs. Status progress polled every 1.5s via `/api/model/status` to update Chart.js.
- Model Serialization:** Trained model saved to `models/nids_lstm_model.keras`; scaler saved to `models/scaler.pkl`.
- Evaluation:** Model tested on test split; evaluation JSON report generated.
- Audit Trail Entry:** Event logged in SQLite `audit_logs` table; Dashboard updated.

Chapter 16: 100 Coordinator & Viva Questions (With Simple Answers)

1. What is an NIDS?

A Network Intrusion Detection System that monitors traffic to identify cyber attacks.

2. What is Zero Trust?

A security model operating on 'Never Trust, Always Verify', checking identity on every request.

3. Why use JWT instead of sessions?

JWT is stateless and scalable, eliminating the need to store session state in server memory.

4. What algorithm is used for password hashing?

Bcrypt with 12 rounds of computational salting.

5. What is RBAC?

Role-Based Access Control, restricting administrative functions to Admin roles.

6. Why use LSTM for NIDS?

Network traffic is sequential time-series data; LSTMs maintain internal memory to catch temporal attack patterns.

7. Why not Decision Trees?

Decision Trees process packets independently, ignoring time sequence context.

8. Why not CNNs?

CNNs excel at 2D spatial grid image data, whereas LSTMs excel at 1D sequential time-series data.

9. What is Flask?

A lightweight Python WSGI web application framework.

10. What database is used?

SQLite managed via Flask-SQLAlchemy ORM.

11. What is LabelEncoding?

Converting text categories (e.g. 'tcp') into integers (e.g. 0).

12. What is StandardScaler?

Transforming features to Mean = 0 and Standard Deviation = 1.

13. Why do we scale features?

To prevent large byte values (100,000) from dominating weight updates over small values (0.5s).

14. Why reshape data to 3D?

Keras LSTM memory cells require 3D tensors `(samples, time\_steps, features)`.

15. What is an Epoch?

One complete forward and backward pass through the entire training dataset.

16. What is Batch Size?

Number of training samples processed before updating neural weights.

17. What optimizer is used?

Adam (Adaptive Moment Estimation), combining momentum and RMSProp.

18. What loss function is used?

Categorical Cross-Entropy for multi-class classification.

19. What is Dropout?

Randomly deactivating a fraction of neurons (20%) during training to prevent overfitting.

20. What is Dense Layer?

A fully-connected neural network layer.

21. What is Softmax?

An activation function converting logits into class probabilities summing to 1.0.

22. What is ReLU?

Rectified Linear Unit (`max(0, x)`), providing fast non-linear activation.

23. What is Accuracy?

Total correct predictions divided by total predictions.

24. Why can high accuracy be misleading?

In imbalanced datasets, predicting only the majority class gives high accuracy but misses all attacks.

25. What is Precision?

Fraction of flagged attacks that were real attacks (`TP / (TP + FP)`).

26. What is Recall?

Fraction of real attacks successfully detected (`TP / (TP + FN)`).

27. What is F1-Score?

Harmonic mean of Precision and Recall (`2 \* P \* R / (P + R)`).

28. What is a Confusion Matrix?

A table comparing actual vs predicted classes showing TP, FP, TN, FN.

29. What is ROC Curve?

Plots True Positive Rate vs False Positive Rate at various thresholds.

30. What is AUC?

Area Under the ROC Curve; values near 1.0 indicate superior class discrimination.

31. What datasets are pre-installed?

NSL-KDD and UNSW-NB15 sample datasets (1,000 rows each).

32. Where are trained models saved?

In ``models/nids_lstm_model.keras``.

33. Where are scalars saved?

In ``models/scaler.pkl`` using Python pickle.

34. Where are evaluation metrics stored?

In ``reports/latest_evaluation_report.json``.

35. What is the Audit Log?

An immutable database table tracking user actions, logins, uploads, and training runs.

36. Can audit logs be exported?

Yes, via one-click CSV export on the dashboard.

37. What Python version is used?

Python 3.11+.

38. What deep learning framework is used?

TensorFlow 2.x with Keras API.

39. What is ``app.py``?

Application Factory initializing Flask, database, and blueprints.

40. What is ``config.py``?

Central configuration file storing secret keys, folder paths, and parameters.

41. How does frontend communicate with backend?

Via asynchronous ``fetch()`` API calls sending JSON payloads and JWT headers.

42. What frontend framework is used?

Bootstrap 5 for responsive CSS layout.

43. What charting library is used?

Chart.js for rendering live loss/accuracy graphs.

44. How is training progress monitored live?

Frontend polls ``/api/model/status`` every 1.5s to fetch current epoch metrics.

45. What is overfitting?

When a model memorizes training noise and performs poorly on unseen test data.

46. How do we prevent overfitting?

Using 20% Dropout layers and Train/Test dataset splitting.

47. What is the default Train/Test split ratio?

80% Training, 20% Testing.

48. What is ``is_selected`` in `DatasetHistory``?

A boolean flag indicating which dataset is currently active for training.

49. Can common users delete datasets?

No, dataset deletion requires Admin role protected by ``@require_role(['Admin'])``.

50. What is the ultimate deliverable of Phase 1?

A serialized ``*.keras`` LSTM model, ``*.pkl`` scaler, ``*.pkl`` label encoder, and metric evaluation report.

51. What is learning rate?

Step size taken by optimizer during gradient descent updates.

52. What happens if learning rate is too high?

Optimizer overshoots optimal weights and training diverges.

53. What happens if learning rate is too low?

Training converges extremely slowly and may get stuck in local minima.

54. What is forward pass?

Passing input data through network layers to produce output predictions.

55. What is backpropagation?

Calculating loss gradients using chain rules and propagating errors backward.

56. What is Gradient Descent?

Optimization algorithm that adjusts weights in the direction of steepest loss descent.

57. What is a tensor?

A multi-dimensional numerical array (e.g. 1D vector, 2D matrix, 3D tensor).

58. Why do we drop duplicate rows?

To prevent the model from memorizing repeated identical samples.

59. How are missing values handled?

Imputed using numerical median statistics.

60. What is a confusion matrix cell FN?

False Negative: An actual attack that the model incorrectly predicted as Normal.

61. Why is False Negative dangerous in cybersecurity?

Because a missed attack allows hackers inside the network undetected.

62. What is False Positive?

A false alarm: Normal traffic incorrectly flagged as an attack.

63. What is True Positive?

An actual attack correctly detected by the model.

64. What is True Negative?

Normal traffic correctly classified as Normal.

65. What is `model.summary()`?

Prints layer names, output shapes, and trainable parameter counts.

66. What is `input_shape=(1, features)`?

Specifies 1 time-step and N feature columns for LSTM input.

67. What is SMOTE?

Synthetic Minority Over-sampling Technique used to balance datasets.

68. What is class weighting?

Assigning higher loss penalties to underrepresented attack classes during training.

69. What is EarlyStopping?

Keras callback that halts training when validation loss stops improving.

70. What is ModelCheckpoint?

Keras callback that saves the best model weights during training.

71. What is secret key in Flask?

Cryptographic key used to sign session cookies and JWT tokens.

72. What is HS256?

HMAC with SHA-256 hash algorithm used to sign JWT tokens.

73. What happens if a JWT token is tampered with?

Signature verification fails and backend rejects request with 401 Unauthorized.

74. What is ORM?

Object-Relational Mapping (SQLAlchemy), bridging Python classes to database tables.

75. Why use SQLite for Phase 1?

Zero configuration, lightweight file-based SQL database ideal for local development.

76. What is `db.session.commit()`?

Persists staged database changes permanently to disk.

77. What is `g.user` in Flask?

Application context object storing current authenticated user payload during request execution.

78. What is `@before_request`?

Flask middleware running before every incoming request to extract JWT tokens.

79. What is a blueprint in Flask?

A modular component grouping related API routes together.

80. What blueprints exist in our app?

`auth_bp`, `dataset_bp`, and `model_bp`.

81. What is `jsonify()`?

Flask helper converting Python dictionaries into JSON HTTP responses.

82. What is `render_template()`?

Renders Jinja2 HTML templates with dynamic Python variables.

83. What is Jinja2?

Templating engine for Python Flask used to build dynamic web pages.

84. What is `url_for()`?

Flask helper generating URLs for view functions based on endpoint names.

85. Why use Bootstrap 5?

Provides modern, responsive CSS grid layout and UI components.

86. What is Chart.js?

JavaScript charting library rendering interactive HTML5 canvas graphs.

87. How does live search work on audit trail?

Debounced JS listener filters table rows based on input text matching.

88. How does CSV export work on dashboard?

JS extracts HTML table content and generates a downloadable CSV data URI link.

89. What is `pickle` in Python?

Standard module for serializing and deserializing Python objects to byte streams.

90. What is `.keras` file format?

Native Keras v3 format storing model architecture, weights, and optimizer state in zip archive.

91. What is `num_classes`?

Total number of unique classification labels in target dataset.

92. What is binary vs multi-class classification?

Binary classifies Normal vs Attack; Multi-class classifies specific attack types (DoS, Probe, R2L, U2R).

93. What is NSL-KDD dataset?

Cleaned benchmark dataset derived from KDD Cup 1999.

94. What is UNSW-NB15 dataset?

Modern network intrusion dataset created by Australian Centre for Cyber Security.

95. How does Phase 1 prepare for Phase 2?

Phase 1 trains and serializes `.keras`` model and `.pkl`` scaler; Phase 2 loads them to classify live packet streams.

96. Will Phase 2 modify Phase 1 code?

No, Phase 2 imports Phase 1 model artifacts modularly without altering Phase 1 codebase.

97. What is a REST API?

Representational State Transfer API using standard HTTP verbs (GET, POST, DELETE) with JSON payloads.

98. What is 401 vs 403 status code?

401 = Unauthorized (Invalid or missing JWT token); 403 = Forbidden (Insufficient RBAC role permissions).

99. How is password salting implemented?

Bcrypt appends a random salt to password before hashing to defeat precomputed rainbow tables.

100. Why are you confident in your project?

Because every module—from Zero Trust authentication to LSTM training and evaluation—is fully implemented, tested, and modularly designed!

Chapter 17: Quick Revision Notes & Exam Cheat Sheet

17.1 Key Formulas Cheat Sheet

- **StandardScaler Normalization:** $z = \frac{x - \mu}{\sigma}$
- **Accuracy:** $\frac{TP + TN}{TP + TN + FP + FN}$
- **Precision:** $\frac{TP}{TP + FP}$
- **Recall:** $\frac{TP}{TP + FN}$
- **F1-Score:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

17.2 Top Coordinator Presentation Tips

1. Emphasize **Zero Trust**: Explain that every API request verifies JWT tokens and RBAC roles.
2. Emphasize **LSTM Sequence Memory**: Explain why LSTMs outperform static decision trees on time-series network data.
3. Highlight **Phase 1 Output Serialization**: Point out that Phase 1 outputs `.keras`` model and `.pkl`` scaler ready for Phase 2 deployment.

Final Mastery Note

You have completed the Phase 1 Study Handbook! Review these 17 chapters and 100 viva questions to deliver a flaw-free presentation.